Name of the Student	**N.Jeevan Sai**
Reg. No	**192525169**
GitHub Link	**https://github.com/Nagellajeevansai/MLA0405-Deep-Learning.git**

SIMATS ENGINEERING

CO3 - ASSESSMENT TOOL 1

Case Study

COURSE NAME: DEEP LEARNING COURSE CODE: MLA0405
SLOT : D

COURSE OUTCOME COVERED

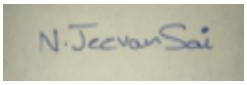
CO3: Analyze and apply convolutional neural network architectures, including convolutional layers, filters, parameter sharing, regularization, AlexNet and ResNet, to develop solutions for image classification and real-world computer vision applications. (L4)

CO Weightage: 25%
Assessment Tool Weightage: 25%

Duration: 90 minutes
Marks: 25

Code of Conduct:

IN.Jeevan Sai(192525169) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student: **Name of the student: N.Jeevan Sai**

Criteria	Conceptual Understanding (2.5)	Linkages (2.5)	Organization (2)	Integration of Literature (2)	Clarity & Presentation (1)	Total (10)
Weightage	***25 %***	***25%***	***20%***	***20 %***	***10 %***	**100%**
Q1						
Q2						

Total						

Signature of the Faculty: ***Total Marks :***

QUESTIONS: 02 **Total Marks: 20**

Case Study Question 1: CNN for Automated Medical Image Classification (10 Marks)
A healthcare organization wants to develop a deep learning system to automatically classify chest X-ray images into **normal** and **disease-affected** categories. The dataset contains thousands of Xray images with variations in size, brightness and image quality. A CNN model is proposed because it can automatically learn important visual features such as edges, textures, and abnormal regions without manually designing features.

Question:

1. Design and explain a suitable **CNN architecture** for this medical image classification system.
2. Describe the **motivation for using CNNs** and explain the role of **convolutional layers, filters, pooling layers, and fully connected layers** in the proposed architecture.
3. Explain how **parameter sharing** reduces the number of trainable parameters compared with a fully connected network.
4. Discuss suitable **regularization techniques such as dropout, data augmentation and batch normalization** to reduce overfitting.
5. Finally, explain whether **AlexNet or ResNet** would be more appropriate for this application and justify your choice based on feature learning, network depth, computational requirements and classification performance.

Case Study Question 2: CNN-Based Autonomous Vehicle Object Recognition (10 Marks)
An autonomous vehicle company is developing a vision system that must identify **cars, pedestrians, traffic signs, bicycles, and road obstacles** from images captured by cameras mounted on the vehicle. The system must recognize objects under different lighting conditions, viewing angles, distances and road environments. The development team is considering **AlexNet and ResNet** architectures for building the CNN-based recognition system.

Question:

1. Analyze how a **CNN architecture** can be designed for this autonomous vehicle application.
2. Explain the purpose of the **input, convolution, activation, pooling, and fully connected/output layers** and describe how convolutional **filters progressively learn lowlevel and high-level features**.
3. Explain the importance of **parameter sharing** in reducing computational complexity and improving translation-related feature detection.
4. Discuss the possible problem of **overfitting** and recommend suitable regularization methods.
5. Compare **AlexNet and ResNet** in terms of architecture, depth, parameter efficiency, training difficulty, vanishing-gradient problems and feature-learning capability.

6. Finally, recommend the most suitable architecture for the autonomous vehicle system and justify your decision based on **accuracy, computational cost and real-time application requirements**.

CASE STUDY 1: CNN FOR AUTOMATED MEDICAL IMAGE CLASSIFICATION

Introduction

A healthcare organization wants to automatically classify chest X-ray images into two categories: Normal and Disease-Affected. Since X-ray images contain important spatial patterns such as edges, textures, shapes, and abnormal regions, a Convolutional Neural Network (CNN) is highly suitable for this task. CNNs can automatically learn useful features directly from images and eliminate the need for manually designed features.

1. Proposed CNN Architecture

A suitable CNN architecture for chest X-ray classification can be designed as follows:

Input X-ray Image → Preprocessing → Convolution + ReLU → Pooling → Convolution + ReLU → Pooling → Convolution + ReLU → Pooling → Flatten → Fully Connected Layer → Dropout → Output Layer

Proposed Architecture

Layer	Function
Input Layer	Accepts resized chest X-ray image, e.g., 224 × 224 × 1
Convolution Layer 1	Detects basic edges and intensity patterns
ReLU Activation	Introduces non-linearity
Max Pooling	Reduces spatial dimensions
Convolution Layer 2	Learns textures and local structures
ReLU	Provides non-linear feature learning
Max Pooling	Reduces feature-map size
Convolution Layer 3	Learns complex structures and abnormal regions
ReLU	Non-linear transformation
Max Pooling	Reduces computational requirements
Flatten	Converts feature maps into a vector
Fully Connected Layer	Performs high-level classification
Dropout	Reduces overfitting
Output Layer	Produces Normal/Disease-Affected prediction

The X-ray images should first be resized to a common resolution and normalized so that variations in image size and brightness do not negatively affect training.

For example, the final output can use two neurons with Softmax:

- Class 0 → Normal

- Class 1 → Disease-Affected

The model can be trained using a suitable loss function such as binary cross-entropy for a twoclass output.

2. Motivation for Using CNNs and Role of Different Layers

CNNs are particularly suitable for medical image classification because they preserve the spatial relationship between pixels and automatically learn hierarchical visual features.

Convolutional Layers

A convolutional layer applies small learnable filters to the input image. These filters move across the image and detect specific patterns.

Early layers generally learn:

- Edges
- Lines
- Corners
- Brightness transitions Deeper layers learn:
- Textures
- Shapes
- Anatomical structures
- Abnormal regions associated with disease

Therefore, CNN feature learning progresses from simple features to complex medical patterns.

Filters

A filter, or kernel, is a small matrix of trainable values. During training, the CNN learns suitable filter values.

For example:

Input X-ray → Edge Filter → Feature Map → Texture Filter → Higher-level Feature Map

Different filters can detect different characteristics of an X-ray.

Pooling Layers

Pooling reduces the spatial dimensions of feature maps while retaining important information.

Max pooling is commonly used to select the maximum value within a small region. Advantages include:

- Reduces computation
- Reduces the number of parameters in later layers
- Provides some tolerance to small image translations
- Helps extract dominant features

Fully Connected Layers

After convolution and pooling, the extracted feature maps are flattened into a onedimensional vector. Fully connected layers combine these learned features and determine the final class.

For example:

Learned features → Fully Connected Layer → Disease probability

Thus, convolutional layers perform feature extraction, while the final layers perform classification.

3. Parameter Sharing

Parameter sharing is one of the major advantages of CNNs.

In a fully connected network, every input pixel may be connected to every neuron. For a 224 × 224 grayscale image:

224 × 224 = 50,176 input pixels

If one fully connected neuron is connected to all pixels, it requires approximately **50,176 weights**, excluding the bias.

A CNN instead uses a small filter such as a **3 × 3 kernel**, which contains only:

3 × 3 = 9 weights

The same 3 × 3 filter is reused at different positions of the image.

Therefore, the CNN:

- Uses far fewer trainable parameters
- Requires less memory
- Reduces computational cost
- Learns the same feature regardless of its position
- Improves generalization

For example, an edge-detection filter learned in one part of an X-ray can also detect a similar edge in another region.

4. Regularization Techniques to Reduce Overfitting

Medical image datasets may contain thousands of images, but a deep CNN can still overfit by memorizing training images instead of learning general disease patterns. **a) Data Augmentation**

Training images can be randomly modified using techniques such as:

- Small rotations
- Horizontal shifts
- Zooming
- Cropping
- Brightness adjustments

This creates additional variations of the training images and improves generalization.

However, augmentation must be medically appropriate. Excessive transformations that change the clinical meaning of an X-ray should be avoided. **b) Dropout**

Dropout randomly disables a proportion of neurons during training.

For example, with a dropout rate of 0.5, approximately half of the selected neurons are temporarily ignored during each training iteration.

This prevents the network from depending too strongly on particular neurons and reduces overfitting.

c) Batch Normalization

Batch normalization normalizes intermediate activations during training.

It can:

- Stabilize training
- Allow faster convergence
- Reduce sensitivity to parameter initialization
- Provide some regularization

d) Additional Measures Other useful techniques include:

- Early stopping
- L2 regularization

- Transfer learning
- Validation-set monitoring

For medical applications, the dataset should also be divided into training, validation, and independent test sets carefully to avoid data leakage.

5. AlexNet vs ResNet Comparison

Feature	**AlexNet**	**ResNet**
Architecture	Relatively simple CNN	Deep CNN using residual blocks
Depth	Relatively shallow	Can be very deep
Feature learning	Good	Excellent hierarchical feature learning
Training deep networks	Easier because network is smaller	Residual connections make deep training easier
Vanishing gradient	More manageable due to shallower depth	Strongly addressed using skip connections
Parameter efficiency	Older architecture with relatively many parameters	Modern variants can provide better accuracy-efficiency trade-offs
Computational requirement	Lower for the original architecture	Depends on variant; deeper versions require more computation
Medical image performance	Good baseline	Generally more suitable for complex image patterns

Why ResNet is More Appropriate

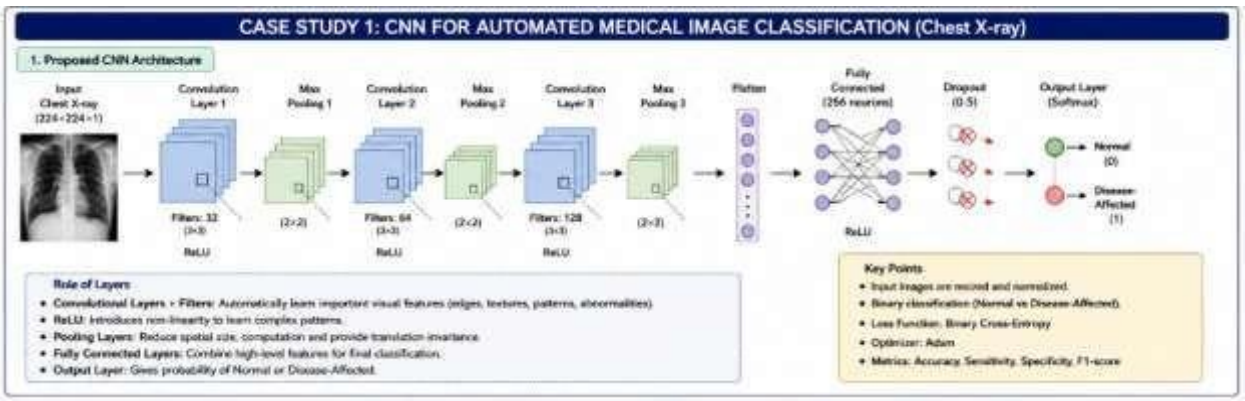
ResNet (Residual Network) is the better choice for this medical classification application.

A key feature of ResNet is the **skip connection**:

Input → Convolution Layers → Output

along with a shortcut:

Input ——————————→ Output



The network learns a residual function instead of requiring every layer to learn a completely new transformation. This makes it easier to train deep networks and reduces the problem of vanishing gradients.Chest X-rays contain complex structures and subtle abnormalities. A deeper ResNet can learn more sophisticated hierarchical features than the original AlexNet.

A practical solution would be a moderate ResNet such as **ResNet-18 or ResNet-34**, especially when computational resources are limited. Transfer learning from a large image dataset can further reduce training requirements.

Conclusion

A CNN is highly suitable for automated chest X-ray classification because it automatically learns relevant visual features from raw images. Convolutional layers extract features, pooling reduces dimensionality, and fully connected layers perform classification. Parameter sharing greatly reduces the number of trainable parameters compared with fully connected networks. Data augmentation, dropout, and batch normalization can reduce overfitting.

Between AlexNet and ResNet, ResNet is recommended because its residual connections support deeper feature learning and help overcome vanishing-gradient problems. A moderate ResNet model provides a strong balance between classification performance and computational requirements, making it suitable for automated medical image classificatio n.

CASE STUDY QUESTION 2: CNN-BASED AUTONOMOUS VEHICLE OBJECT RECOGNITION

Introduction

An autonomous vehicle must recognize objects such as cars, pedestrians, bicycles, traffic signs, and road obstacles from camera images. The system must work under different lighting conditions, viewing angles, object distances, and road environments.

CNNs are suitable because they can automatically learn visual features from images and classify objects based on their learned representations. For an actual autonomous vehicle, the system may ultimately require object detection rather than only image classification because it must identify both the object and its location in the image.

1. CNN Architecture for Autonomous Vehicle Recognition A suitable CNN-based architecture can be represented as:

Camera Image → Preprocessing → Convolution → ReLU → Pooling → Convolution → ReLU → Pooling → Deeper Convolution Layers → Feature Extraction → Classification/Detection Head → Object Class and Location

The input image can be resized to a fixed resolution and normalized before being given to the network.

The system should be trained using images containing different:

- Cars
- Pedestrians
- Bicycles
- Traffic signs
- Road obstacles

The dataset should also contain variations in:

- Day and night conditions
- Rain and fog
- Bright and low-light scenes
- Different camera angles
- Near and distant objects
- Urban and highway environments

For basic classification, the output layer can provide class probabilities. For a complete autonomous-driving system, a detection architecture would additionally provide bounding boxes and class labels.

2. Purpose of CNN Layers and Progressive Feature Learning Input Layer

The input layer receives camera images, for example:

224 × 224 × 3 where 3 represents the RGB channels.

Preprocessing can include:

- Resizing
- Pixel normalization
- Appropriate image augmentation

Convolutional Layer

Convolution applies filters to the image to extract local patterns.

The first layers may detect:

- Edges
- Lines
- Corners
- Color transitions

The deeper layers combine these features to learn:

- Shapes
- Wheels
- Human body structures
- Traffic-sign patterns
- Vehicle structures

Activation Function

The **ReLU (Rectified Linear Unit)** activation introduces non-linearity. It is commonly expressed as:

ReLU(x) = max(0, x)

This allows the CNN to learn complex relationships rather than only simple linear patterns.

Pooling Layer

Pooling reduces the size of feature maps.

For example, max pooling selects the strongest activation from a local region. It helps:

- Reduce computation
- Reduce feature-map dimensions
- Retain important information
- Provide some tolerance to small translations

Fully Connected/Output Layer

After feature extraction, the learned representation is used for classification.

For example:

Feature representation → Classifier → Car / Pedestrian / Bicycle / Traffic Sign / Obstacle

For object detection, the output additionally needs information about object location, such as bounding-box coordinates.

Progressive Feature Learning

CNN feature learning can be represented as:

Pixels → Edges → Textures → Shapes → Object Parts → Complete Objects For example:

Edges → Curves → Wheels + Body → Car

Similarly:

Edges → Body Parts → Human Shape → Pedestrian

This hierarchical feature learning is one of the major reasons CNNs are effective for autonomous vehicle vision.

3. Importance of Parameter Sharing

Parameter sharing means that the same convolutional filter is applied at different locations of an image.

Suppose a 3 × 3 filter is used. It has only:

3 × 3 = 9 weights

These weights are reused across the entire image rather than creating different weights for every pixel location.

This provides several advantages:

Reduced Number of Parameters

A fully connected network would require a very large number of connections for highresolution images. CNNs dramatically reduce this number using local connectivity and shared filters.

Lower Computational Complexity

Fewer trainable parameters result in lower memory usage and reduced computation.

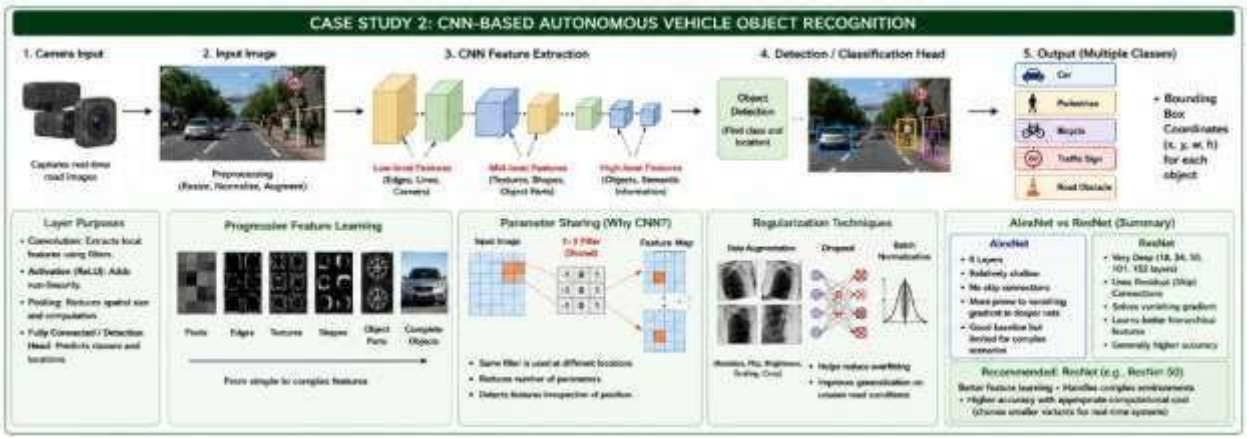
Translation-Related Feature Detection

If a car wheel appears on the left side of an image or the right side, the same learned filter can detect the wheel pattern.

Therefore, the CNN can recognize useful features even when their position changes. This is particularly important for autonomous vehicles because objects can appear at different positions in the camera frame.

4. Overfitting and Regularization

Autonomous vehicle datasets can contain millions of images, but the model can still overfit to particular roads, cameras, weather conditions, or environments. Suitable regularization methods include:



Data Augmentation

Images can be modified using:

- Rotation
- Cropping
- Scaling
- Translation
- Brightness variation

- Contrast variation
- Weather-related augmentation

This helps the network handle different real-world conditions. **Dropout**

Dropout randomly disables selected neurons during training. This prevents the network from becoming excessively dependent on specific neurons.

Batch Normalization

Batch normalization stabilizes intermediate activations and can improve training speed and generalization.

Weight Regularization

L2 regularization can penalize excessively large weights and help prevent over-complex models.

Early Stopping

Training can be stopped when validation performance stops improving. This prevents unnecessary training and overfitting.

A separate validation and test dataset should be used to evaluate the model fairly.

5. AlexNet vs ResNet

Feature	**AlexNet**	**ResNet**
Basic design	Traditional deep CNN	CNN with residual/skip connections
Depth	Relatively shallow	Can be very deep
Feature learning	Good	Strong hierarchical feature learning
Training difficulty	Easier because it is smaller	Deep networks are easier to optimize due to skip connections
Vanishing gradients	More manageable due to limited depth	Residual connections significantly reduce the difficulty of training deep networks
Parameter efficiency	Older architecture and relatively parameter-heavy	Many variants offer strong accuracy relative to model size
Accuracy	Good baseline	Generally better for complex visual recognition
Computational cost	Lower than very deep ResNet models	Depends strongly on the ResNet version
Real-time suitability	Possible	Moderate/smaller ResNet variants are suitable
Modern applications	Less commonly preferred	Widely used as a strong CNN backbone

Feature-Learning Capability

AlexNet can learn useful features, but its architecture is relatively old and shallow compared with modern residual networks.

ResNet can contain many more layers while remaining trainable because of skip connections.

A simplified residual block is:

Input → Convolution → ReLU → Convolution → Addition → ReLU

The original input is passed through a shortcut and added to the output of the convolutional path.

This helps gradients propagate through the network during training.

6. Recommended Architecture for Autonomous Vehicles

For the autonomous vehicle system, **ResNet is generally the better choice**. The main reasons are:

Accuracy

Autonomous vehicles require reliable recognition of pedestrians, vehicles, signs, bicycles, and obstacles. ResNet's deeper feature-learning capability allows it to learn more complex visual patterns.

Feature Learning

ResNet can learn hierarchical features ranging from simple edges to complex object structures. For example:

Edges → Shapes → Wheels/Body → Vehicle and:

Edges → Curves → Human Body Parts → Pedestrian

Training Difficulty

Very deep networks can suffer from vanishing gradients. ResNet addresses this using residual/skip connections, making deeper models easier to train.

Computational Cost

Although a very deep ResNet can be computationally expensive, the solution does not necessarily require the largest ResNet model.

A practical system can use a smaller ResNet variant such as ResNet-18 or ResNet-34 when computational resources and latency are important.

For real-time autonomous driving, the architecture should be selected based on a balance between:

Accuracy + Inference Speed + Memory Usage + Hardware Capability

If the vehicle has powerful GPU/AI hardware, a larger ResNet can be considered. For embedded hardware with strict latency constraints, a smaller ResNet or an optimized lightweight detection network would be preferable.

Final Recommendation

ResNet is recommended over AlexNet for the autonomous vehicle application because it provides stronger feature-learning capability, better support for deep networks, and improved training through residual connections.

However, the largest possible ResNet should not automatically be selected. A moderate model such as **ResNet-18/34** can provide a better trade-off between accuracy and real-time computational requirements.

For a complete autonomous-driving solution, the CNN backbone should ultimately be integrated with an object-detection framework so that the vehicle can determine both what the object is and where it is located.

Conclusion

CNNs provide an effective foundation for autonomous vehicle vision because they automatically learn hierarchical features from camera images. Convolutional filters identify lowlevel patterns first and progressively learn high-level object features. Parameter sharing reduces the number of trainable parameters and allows the same feature detector to operate across different image locations.

Regularization methods such as data augmentation, dropout, batch normalization, weight regularization, and early stopping help improve generalization. Between AlexNet and ResNet, ResNet is the more suitable architecture because of its superior feature-learning capability and

residual connections. A suitably sized ResNet can achieve a strong balance between accuracy, computational cost, and real-time performance.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding ; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, wellstructured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand